Assignment (Questions): Penguins

Instructor: Dr. Khaled Mohamad

Your Role

You are a junior data analyst at a wildlife research center. Your team collected measurements of penguins from different islands. Management wants you to **clean the data**, **explore patterns**, and finally build a **simple model** to predict the penguin species.

The work is structured as follows:

- Part A: Understanding the Data
- Part B: Filtering, Sorting, Aggregation
- Part C: Visualization & Interpretation
- Part D: Apply ML model

Import Required Modules, Libraries, and Functions in the Cell Below

In [22]:
```
--


# You need the following functions (copy them)
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

Part A: Understanding the Dataset

Q1: Load the `penguins` dataset from seaborn into a DataFrame named `df` and display the first 5 rows.

In [ ]:
```

```

Q2: Print the following:

1. dataset shape,
2. column names,
3. data types.

In [ ]:

In [ ]:

In [ ]:

Q3: Check for missing values in each column:

In [ ]:

Q4: Create a **cleaned copy** called `df_clean` by:

1. Removing fully duplicated rows (if any)
2. Handling missing values:
   - For numeric columns: fill missing values with the **median**
   - For categorical columns: fill missing values with the **mode**

Then verify there are **no missing values** remaining.

In [ ]:

In [9]:
```
# 1) Drop duplicates
```

In [11]:
```
# 2) Fill missing values
```

In [ ]:

In [14]:
```
# Verify
```

Part B: Filtering, Sorting, and Aggregation

Q5: Filter the dataset to include only penguins from the island `"Biscoe"`.

Store the result in `biscoe_df` and print its shape.

In [ ]:

Q6: Find all penguins with:

- `body_mass_g` greater than the **overall mean** body mass
- AND `sex` equal to `"Male"`

Store the result in `heavy_males` and display the first 5 rows.

In [ ]:

```
```

Q7: Sort `df_clean` by:

1. `body_mass_g` (descending)
2. then `bill_length_mm` (ascending)

Show the top 10 rows.

In [ ]:

```
```

Q8: Compute the following aggregations:

1. Mean `body_mass_g` per `species`. Sort values in descending order.
2. Mean `bill_length_mm` per (`species` , `sex`)
3. Count of records per `island`

In [19]:

```
# 1) Mean body mass per species
```

In [21]:

```
# 2) Mean bill length per (species, sex)
```

In [23]:

```
# 3) Count per island
```

Q9: Feature engineering:

Create a new column `bill_ratio` = `bill_length_mm` / `bill_depth_mm`.

1. Add it to `df_clean`
2. Show summary statistics for `bill_ratio`
3. Identify which species has the **highest average bill_ratio**

In [25]:

```
#1) Add it to `df_clean`
```

In [27]:

```
#2) Show summary statistics for `bill_ratio`
```

In [29]:

```
#3) Identify which species has the **highest average bill_ratio**
```

Part C: Visualization and Interpretation

Q10: Create the following plots using `df_clean`:

1. Bar plot: count of penguins per species
2. Scatter plot: `bill_length_mm` vs `bill_depth_mm` (color by species)

In [31]:
```
# 1) Count per species (bar)
```

In [33]:
```
# 2) Scatter: bill length vs bill depth (color by species)
```

Q11: Create a box plot of `body_mass_g` by species.

Then write **2–3 sentences** interpreting what you observe.

In [ ]:
```

```

Interpretation

-
-

Q12: Compute a correlation matrix for numeric features and visualize it as a heatmap.

In [ ]:
```

```

Part D: Machine Learning

Q13: Define:

- target `y` = `species`
- features `X` = all other columns EXCEPT `species`

Then split into train/test using:

- test_size = 0.2
- random_state = 42
- stratify by y

In [ ]:
```

```

Q14: Preprocessing Data for ML Training

This step has already been completed for you. It will be explicitly explained and discussed in detail during class.

We did the following:

- Handle missing values (even if we cleaned, do it properly in pipeline)
- One-hot encode categorical columns (`sex` , `island`)
- Standardize numeric columns

Note: You should copy paste this code and run it in the same order as the previous questions.

In [102…

```
# Identify columns by type
numeric_features = X.select_dtypes(include=[np.number]).columns.tolist()
categorical_features = X.select_dtypes(exclude=[np.number]).columns.tolist()

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocess = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features),
    ]
)
```

Q15: Train models:

1. **KNN Classifier** with `n_neighbors=5`

In [114…

```
# 1) KNN pipeline
knn_model = Pipeline(steps=[
    ("preprocess", preprocess),
    ("model", KNeighborsClassifier(n_neighbors=5))
])
```

In [41]:

```
#Train the model using the fit method
```

In [43]:

```
# Predict on the X_test
```

In [45]:

```
#calculate and print the accuracy based on the prediction of the X_test
```

2. **Logistic Regression** (multiclass) with `max_iter=2000`

In [123…
```
# 2) Logistic Regression pipeline
logreg_model = Pipeline(steps=[
    ("preprocess", preprocess),
    ("model", LogisticRegression(max_iter=2000))
])
```

In [47]:
```
# Train the model using the fit method
```

In [49]:
```
# Predict on the X_test
```

In [51]:
```
#calculate and print the accuracy based on the prediction of the X_test
```

Q16: Conclude based on the accuracy results

Answer:

In [ ]:
```

```